Estimating Token Cost and Latency for an LLM Feature

Introduction

Before a single line of code runs in production, a feature built on an LLM already has a cost and a speed — they just haven't been measured yet. Estimating both ahead of time turns "let's see how it goes" into a number you can defend to a product owner or a finance review. This reading walks through how to estimate token cost and latency for a feature before it ships, using nothing more than the prompt design and a provider's published rates.

Two questions, one starting point: token counts

Both cost and latency estimates start from the same input: how many tokens move through the request. Cost is driven by input tokens plus output tokens. Latency is driven mostly by output tokens, since most providers generate one token at a time.

Every estimate — cost or latency — starts by breaking a request into its two token counts: what goes in, and what comes back out. Get those two numbers roughly right, and both estimates follow from there.

Estimating input tokens



Estimating output tokens

Bounded output

A classification label, a short structured field, a yes/no. Output length is predictable and small.

Open-ended output

A summary, an explanation, a generated paragraph. Output length varies and needs a working estimate, not a guess.

Output tokens are harder to pin down than input tokens because they depend on what the model chooses to generate. A feature that classifies text into a fixed set of labels has a nearly fixed, small output. A feature that drafts a paragraph does not — the estimate here has to come from testing representative prompts and taking a realistic average (or a max_tokens cap) rather than assuming the shortest possible answer.

Turning token counts into a cost estimate

Step 1

Input tokens × rate

`600 × $1/1M`

$0.0006

Step 2

Output tokens × rate

`250 × $4/1M`

$0.0010

Step 3

Sum = cost/request

`$0.0006 + $0.0010`

$0.0016

Step 4

× requests/day

`$0.0016 × 5,000`

$8.00

Cost per request

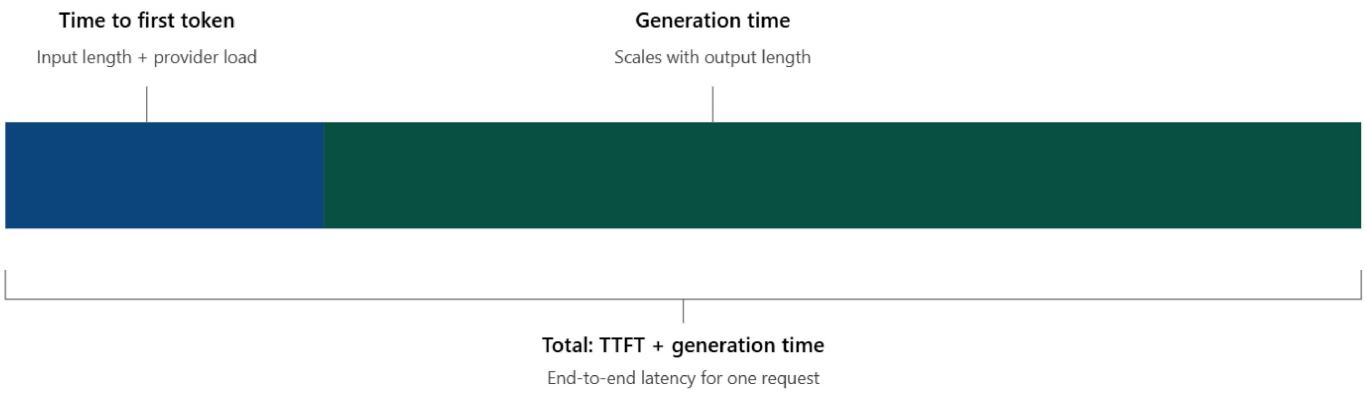
$0.0016

Total daily feature cost

$8.00

Once input and output token estimates exist, cost is arithmetic — but the arithmetic has to use both rates separately, since output tokens are usually priced higher than input tokens. A feature that looks cheap per request can still be expensive at scale if it's called thousands of times a day; the daily or monthly total is the number that actually matters for a budget conversation, not the per-call figure alone.

Turning token counts into a latency estimate



Latency has two distinct pieces that behave differently. Time to first token is mostly a function of how much the model has to read before it starts responding — a longer prompt or conversation history adds here. Generation time scales with how much the model has to write — a feature with a long, open-ended output will feel slower than one with a short, bounded output, even if both use the same model and the same input length. Streaming a response can make an application feel faster without actually changing the total generation time, since the user sees tokens as they're produced rather than waiting for the full response.

Where estimates go wrong

> The most common estimation mistake isn't guessing the wrong number of tokens — it's forgetting that fixed input (system prompt, few-shot examples, growing conversation history) is paid and waited on every single request, not just once per feature.

A cost or latency estimate built only from "the part the user typed" will consistently undershoot both, because it leaves out the fixed overhead every request carries. The fix isn't a more precise formula — it's remembering to count the whole request, not just its most visible part.

Summary

Same two numbers

Both cost and latency estimates start from the same two numbers: estimated input tokens and estimated output tokens.

Hidden overhead

Input tokens include fixed overhead — system prompt, few-shot examples, history — that's easy to forget when estimating.

Different scaling

Cost scales with both token counts at their respective rates. Latency scales mostly with output length, plus a fixed startup delay.

Check against reality

Estimating before shipping turns a guess into a number that can be checked against real usage once the feature is live.